Case Study Report

On Case Study 153: Library Search System (C++)

By [Kanishk Singh]

Roll NO [150096725104]

Semester 2 / Jeff Bezos

Index

1. Introduction to the Case Study
2. Problem Statement / Case Background (Abstract)
3. Problem Statement / Case Study Design
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

5. Problem Statement / Case Study Implementation Details and Snapshots
6. Problem Statement / Case Study Results and Conclusion
7. References

1. Introduction to the Case Study

Libraries manage vast amounts of data, making it crucial to have efficient systems for finding specific resources. Traditional manual searching is time-consuming and prone to human error. This case study focuses on designing and implementing a digital Library Search System that allows users to automatically look up books based on title or author using an interactive command-line interface.

The project demonstrates how data can be structured using array-based handling, manipulated for case-insensitive matching, and retrieved using optimized searching algorithms. It highlights the importance of data formatting, efficient algorithm selection, and user-friendly program flow in building a practical local application.

2. Problem Statement / Case Background (Abstract)

Background Standard linear retrieval methods check data one by one, which becomes inefficient as a library's dataset grows. Implementing structured data handling and advanced searching algorithms allows systems to locate information rapidly and adapt better to user input variations, such as inconsistent capitalization.

Abstract This case study presents the design and implementation of a Library Search System using C++. A predefined dataset of books and their corresponding authors is stored using array-based data handling. The system employs a custom preprocessing function to convert all text to lowercase, ensuring accurate, case-insensitive matching.

Algorithms such as Binary Search and Linear Search are implemented to handle title and author lookups, respectively. The system is evaluated based on the accuracy of its search results, its ability to handle case-insensitive matching, and data retrieval efficiency. The implementation includes an interactive menu loop, robust input handling, and clear formatted output for the user.

3. Case Study Design

The Library Search System is designed as a modular program with the following operational stages:

- **Data Initialization: Use a predefined dataset of books stored within an array of structures, containing string variables for titles and authors.**
- **Data Sorting (Optimization): Sort the array alphabetically by book title at runtime to enable highly efficient search algorithms.**
- **User Interface Flow: Display a continuous menu allowing the user to choose between searching by title, searching by author, or exiting the program.**
- **Input Preprocessing: Clean raw user input by clearing input buffers and converting target strings to lowercase before comparison.**
- **Search Execution: Apply Binary Search for exact title matches and Linear Search to find multiple books by the same author.**
- **Result Display: Output formatted success messages with book details, or clear "not found" alerts if the data does not exist in the array.**

4. Methods & Algorithms Technology Applied

Key methods and algorithms:

- **Array-based Data Handling: Utilizing C++ struct arrays to bind related data points (titles and authors) together in contiguous memory.**
- **Case-Insensitive String Manipulation: Utilizing std::transform and tolower to normalize data for accurate comparison.**
- **Binary Search Algorithm: A divide-and-conquer search technique applied to sorted arrays to achieve $O(\log N)$ time complexity for title retrieval.**
- **Linear Search Algorithm: A sequential check used for author lookups to ensure all matching records are found, even if an author has multiple entries.**

Technology Stack Used for the Case

- **Programming Language: C++.**
- **Libraries and Tools: <iostream> for input/output, <string> for text variables, <algorithm> for sorting and transformations, and <cctype> for character handling.**
- **Environment: VS Code and macOS Terminal (Clang/g++ compiler).**

5. Implementation Details and Snapshots

The implementation is organized within a single executable C++ file, structured logically into functional blocks:

- **Data Structures: Definition of the Book struct to hold the dataset schema.**
- **Helper Functions: The toLowerCase() function to handle input normalization.**
- **Search Logic: The standalone binarySearchTitle() function engineered to calculate midpoints and eliminate halves of the array during title searches.**
- **Main Application Loop: A do-while loop containing a switch statement that acts as the user interface, continually prompting for input until the exit command is triggered.**

```
TERMINAL                                                    zsh
kanishksingh@kanishks-MacBook-Air C ++ Case Study  % g++ library.cpp
 -o library
kanishksingh@kanishks-MacBook-Air C ++ Case Study  % ./library

--- Library Search System ---
1. Search by Title (Fast Binary Search)
2. Search by Author
3. Exit
Enter your choice (1-3): 2
Enter Author Name to search: harper lee

Book Found Successfully!
Title: To Kill a Mockingbird | Author: Harper Lee
```

6. Results and Conclusion

Findings

- **Sorting the array and applying Binary Search significantly improves data retrieval efficiency compared to standard sequential searching.**
- **Implementing strict input buffering (cin.ignore()) and getline() is mandatory for accurately capturing multi-word titles and names.**
- **Converting both the dataset parameters and the user input to lowercase prior to evaluation ensures 100% case-insensitive matching accuracy.**

Conclusion This case study demonstrates a complete and optimized pipeline for building a Library Search System using C++. It successfully integrates array-based data handling, dynamic input preprocessing, and efficient algorithm implementation into a cohesive command-line application, fulfilling all evaluation criteria and providing a strong foundation for software development logic.

7. References

- **Official C++ documentation for standard libraries (<iostream>, <string>, <algorithm>).**
- **Academic tutorials on Binary Search logic and time complexity analysis**